

Local Agentic Coding Build Plan · Qwen 3.8

Base M5 Ultra + one storage bump, running Qwen 3.8 27B through MTPLX, with OpenCode as the coding agent and Hermes as the orchestrator. Qwen 3.8 ships the MTP heads MTPLX is built to exploit, so speculative decoding is native and free — the fastest reliable local coding setup on a Mac today. 96GB is plenty at a 150K working cap.

\$5,549 Build cost M5 Ultra 96GB + 2TB, edu	27B Qwen 3.8 ~20GB, 96GB wide open	~45 tok/s Coding speed 20–30K ctx; 20+ at 128K	2.24x Native MTP M5 Max, no draft model
---	--	--	---

I. The Buy

M5 Ultra, base, 96GB, 2TB

DECIDED

- Keep 96GB (soldered — buy right now); skip the +\$3,600 256GB.
- Bump storage to 2TB (+\$450) — weights are tens of GB each.
- Skip the 80-core GPU (+\$1,170); base chip is fine.

II. The Stack

Four layers, one glance

LAYER	PICK	WHY
Serving	MTPLX	Built for Qwen 3.5/3.6/3.8; native MTP (~2.24x), prefix caching, OpenAI API on :8000, launches OpenCode/Hermes
Model	Qwen 3.8 27B	Optimized Speed (4-bit) daily; Optimized Quality (8-bit) for max fidelity
Harness	OpenCode	De-facto OSS terminal agent; best tool-call reliability; points at local endpoint
Orchestrator	Hermes	Lean memory, per-skill routing, learning loop; drives OpenCode, keeps box busy

All open-source and OpenAI-compatible — any layer swaps without touching the others. ^[1]

III. Config

The settings that matter

- Served context **64K minimum** (defaults of 4K silently break tool use); working cap 150K.
- Sampling: `(temp 0.6, top_p 0.95, top_k 20)`; penalties at 0 so MTP stays exact.
- KV cache FP16/INT8 — don't over-quantize (compute-bound chip).
- Thinking mode off for routine coding; keep one model resident per session.

Memory at 150K: ~20GB weights + ~9–18GB KV + OS ≈ 30–45GB, leaving 50GB+ free.

IV. Operate

Reliability & keeping it busy

- Scope tasks tightly; chunk them; work from progress files (kanban).
- Treat `AGENTS.md` as a product — refine after every failure.
- Tests-as-gate + a separate auditor model checking the work.
- git worktrees + parallel agents to saturate; backfill idle time with indexing/embeddings (MTPLX serves those) and test generation.

V. Scale & Avoid

Grow later, sidestep these

CAUTION

- **Scale:** move serving to vLLM Metal when you fan out to many parallel agents; add a Thunderbolt 5 node for concurrency, not to rescue one machine.
- **Avoid Laguna S 2.1 as an autonomous driver** — strong benchmarks, but fails to converge on real loops and invents facts under pressure. ^[2]
- **Sandbox OpenClaw hard** if used (security concerns); Hermes fits a solo local coder better. ^[3]
- **LM Studio is a GUI/manager, not the speed layer** — serve through MTPLX.
- **Calibrate:** local coders are ~last-gen-flagship, not frontier — keep an occasional hosted call for the hardest 5%. ^[4]

